

Text Book: Introduction to the Design and Analysis of Algorithms

Author: Anany Levitin 2 nd Edition

Unit-4

3. Input Enhancement in String Matching – Horspool’s algorithm

String matching involves finding an occurrence of a given string of m characters called the **pattern** in a longer string of n characters called the **text**.

Several faster algorithms have been discovered. Most of them exploit the input-enhancement idea: preprocess the pattern to get some information about it, store this information in a table, and then use this information during an actual search for the pattern in a given text. This is exactly the idea behind the two best known algorithms of this type: the Knuth-Morris-Pratt algorithm and the Boyer-Moore algorithm. The principal difference between these two algorithms lies in the way they compare characters of a pattern with their counterparts in a text: the Knuth-Morris-Pratt algorithm does it left to right, whereas the Boyer-Moore algorithm does it right to left.

Although the underlying idea of the Boyer-Moore algorithm is simple, its actual implementation in a working method is less so. Therefore, we start our discussion with a simplified version of the Boyer-Moore algorithm suggested by R. Horspool . In addition to being simpler, Horspool’s algorithm is not necessarily less efficient than the Boyer-Moore algorithm on random strings.

Horspool’s Algorithm

Consider, as an example, searching for the pattern **BARBER** in some text:

$$S_0 \dots C \dots S_{n-1}$$

B A R B E R

Starting with the last **R** of the pattern and moving right to left, we compare the corresponding pairs of characters in the pattern and the text. If all the pattern’s characters match successfully, a matching substring is found. Then the search can be either stopped altogether or continued if another occurrence of the same pattern is desired. If a mismatch occurs, we need to shift the pattern to the right.

Clearly, we would like to make as large a shift as possible without risking the possibility of missing a matching substring in the text. Horspool's algorithm determines the size of such a shift by looking at the character c of the text that is aligned against the last character of the pattern. This is the case even if character c itself matches its counterpart in the pattern.

In general, the following four possibilities can occur.

Case 1 If there are no c 's in the pattern—e.g., c is letter **S** in our example—we can safely shift the pattern by its entire length (if we shift less, some character of the pattern would be aligned against the text's character c that is known not to be in the pattern):

s_0	...		S		s_{n-1}
			X		
		B A R B E R			
			B A R B E R		

Case 2 If there are occurrences of character c in the pattern but it is not the last one there—e.g., c is letter **B** in our example—the shift should align the rightmost occurrence of c in the pattern with the c in the text:

s_0	...		B		s_{n-1}
			X		
		B A R B E R			
		B A R B E R			

Case 3: If c happens to be the last character in the pattern but there are no c 's among its other $m - 1$ characters—e.g., c is letter **R** in our example—the situation is similar to that of Case 1 and the pattern should be shifted by the entire pattern's length m :

$$\begin{array}{ccccccccccc}
 s_0 & \dots & & & M & E & R & & \dots & s_{n-1} \\
 & & & & X & \parallel & \parallel & & & \\
 & & L & E & A & D & E & R & & \\
 & & & & & & & L & E & A & D & E & R
 \end{array}$$

Case 4: Finally, if c happens to be the last character in the pattern and there are other c 's among its first $m - 1$ characters—e.g., c is letter **R** in our example—the situation is similar to that of Case 2 and the rightmost occurrence of c among the first $m - 1$ characters in the pattern should be aligned with the text's c :

$$\begin{array}{ccccccccccc}
 s_0 & \dots & & & A & R & & \dots & s_{n-1} \\
 & & & & X & \parallel & & & \\
 & & R & E & O & R & D & E & R & & \\
 & & & & R & E & O & R & D & E & R
 \end{array}$$

Formula To compute Shift Table

$$t(c) = \begin{cases} \text{the pattern's length } m, & \text{if } c \text{ is not among the first } m - 1 \text{ characters of the pattern;} \\ \text{the distance from the rightmost } c \text{ among the first } m - 1 \text{ characters} & \\ \text{of the pattern to its last character, otherwise.} & \end{cases} \quad (7.1)$$

For example, for the pattern **BARBER**, all the table's entries will be equal to 6, except for the entries for E, B, R, and A, which will be 1, 2, 3, and 4, respectively. Here is a simple algorithm for computing the shift table entries. Initialize all the entries to the pattern's length m and scan the pattern left to right repeating the following step $m - 1$ times: for the j^{th} character of the pattern ($0 \leq j \leq m - 2$),

overwrite its entry in the table with $m - 1 - j$, which is the character's distance to the last character of the pattern. Note that since the algorithm scans the pattern from left to right, the last overwrite will happen for the character's rightmost occurrence exactly as we would like it to be.

ALGORITHM *ShiftTable*($P[0..m - 1]$)

```
//Fills the shift table used by Horspool's and Boyer-Moore algorithms
//Input: Pattern  $P[0..m - 1]$  and an alphabet of possible characters
//Output:  $Table[0..size - 1]$  indexed by the alphabet's characters and
//       filled with shift sizes computed by formula (7.1)
for  $i \leftarrow 0$  to  $size - 1$  do  $Table[i] \leftarrow m$ 
for  $j \leftarrow 0$  to  $m - 2$  do  $Table[P[j]] \leftarrow m - 1 - j$ 
return  $Table$ 
```

Horspool's algorithm

Step 1 For a given pattern of length m and the alphabet used in both the pattern and text, construct the shift table as described above.

Step 2 Align the pattern against the beginning of the text.

Step 3 Repeat the following until either a matching substring is found or the pattern reaches beyond the last character of the text. Starting with the last character in the pattern, compare the corresponding characters in the pattern and text until either all m characters are matched (then stop) or a mismatching pair is encountered. In the latter case, retrieve the entry $t(c)$ from the c 's column of the shift table where c is the text's character currently aligned against the last character of the pattern, and shift the pattern by $t(c)$ characters to the right along the text.

ALGORITHM *HorspoolMatching*($P[0..m-1]$, $T[0..n-1]$)

```

//Implements Horspool's algorithm for string matching
//Input: Pattern  $P[0..m-1]$  and text  $T[0..n-1]$ 
//Output: The index of the left end of the first matching substring
//         or  $-1$  if there are no matches
ShiftTable( $P[0..m-1]$ )    //generate Table of shifts
 $i \leftarrow m-1$            //position of the pattern's right end
while  $i \leq n-1$  do
     $k \leftarrow 0$          //number of matched characters
    while  $k \leq m-1$  and  $P[m-1-k] = T[i-k]$  do
         $k \leftarrow k+1$ 
    if  $k = m$ 
        return  $i-m+1$ 
    else  $i \leftarrow i + \text{Table}[T[i]]$ 
return  $-1$ 

```

EXAMPLE : As an example of a complete application of Horspool's algorithm, consider searching for the **pattern BARBER** in a text that comprises English letters and spaces (denoted by underscores). The shift table, as we mentioned, is filled as follows:

character c	A	B	C	D	E	F	...	R	...	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

The actual search in a particular text proceeds as follows:

```

J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
B A R B E R           B A R B E R
      B A R B E R       B A R B E R
          B A R B E R           B A R B E R

```